

RAPPORT DE STAGE

Étude et mise en œuvre d'un système de communication à distance basé sur LoRaWAN : Rucher connecté

Bocar KANTE

Master 1 Électronique, Énergie Électrique et Automatique
Parcours Systèmes et Microsystèmes Embarqués

18 mai 2026 – 31 août 2026

Tuteur : M. Thierry PERISSE

Table des matières

1	Introduction et Présentation du Projet	3
1.1	Contexte	3
1.2	Objectifs du système et spécification	3
1.3	Architecture globale	4
2	Développement Firmware STM32	5
2.1	Matériel utilisé	5
2.1.1	Description technique des modules	5
2.2	Schéma de câblage	7
2.3	Configuration du fichier .ioc (STM32CubeIDE / CubeMX)	8
2.4	Architecture logicielle du firmware	9
2.4.1	Description du main	10
3	LoRaWAN : Communication Wio-E5	12
3.1	Interface physique STM32 $\longleftrightarrow$ Wio-E5	12
3.2	Mécanisme de communication AT	12
3.2.1	Fonction AT_Send()	12
3.2.2	Séquence de jointure OTAA (implémentation réelle)	13
3.3	Mode test / point-à-point (débugage RF)	13
3.4	Réception et décodage des trames descendantes	14
3.5	Transmission des données applicatives	14
3.6	Étude des caractéristiques de la modulation LoRa	14
3.6.1	Principe de la modulation LoRa	15
3.6.2	Étude du Spreading Factor (SF)	16
3.6.3	Étude de la bande passante (BW)	16
3.6.4	Étude du Coding Rate (CR)	16
3.6.5	Étude de la puissance d'émission et cadre réglementaire	17
3.6.6	Débit binaire utile et impact conjoint de SF, BW et CR	17
3.6.7	Étude du mécanisme d'adaptation dynamique (ADR)	18
3.7	Synthèse de l'étude et paramètres retenus	18
4	Gateway LoRaWAN(SenseCAP M2)	20
4.1	Présentation du matériel	20
5	The Things Network (TTN V3)	21
5.1	Configuration du compte et de l'application	21
5.2	Procédure OTAA	21
5.3	Payload Formatter (décodeur TTN)	21
6	Stack Backend : MQTT, Node-RED, InfluxDB, Grafana	22
6.1	Architecture déployée	22
6.2	Connexion MQTT TTN $\rightarrow$ Node-RED	22

6.3	Pipeline Node-RED	22
6.4	InfluxDB 2.7	23
6.5	Grafana	24
7	Prise en main de l'analyseur FPL - Étude de la couche physique LoRa	25
7.1	Contexte	25
7.2	Choix des paramètres de configuration de l'analyseur	25
7.2.1	Fréquence centrale et span	26
7.2.2	Résolution en fréquence (RBW) et compromis temps-fréquence	26
7.2.3	Temps d'acquisition	26
7.2.4	Déclenchement (trigger)	27
7.2.5	Niveau de référence et atténuation	27
7.2.6	Fréquence d'échantillonnage (mode IQ / capture transitoire)	27
7.2.7	Exemple de capture : Mode Transient Analysis (logiciel VSE)	27
7.3	Observations réalisées sur le signal LoRa	29
7.3.1	Interprétation de la structure préambule/payload observée	29
8	Bilan et Perspectives	31
8.1	État d'avancement	31
8.2	Difficultés rencontrées	31
8.3	Bilan énergétique	32
8.4	Perspectives	33
8.5	Bilan général	33
A	Annexe — Fonctions de décodage Node-RED et TTN	35
A.1	Décodage des données reçues sur TTN	35
A.2	Extraction de la tension batterie	36
A.3	Extraction de la valeur de détection de mouvement	36
A.4	Extraction de la valeur de détection de la masse	36
A.5	Extraction de la valeur de détection de l'humidité	37

1. Introduction et Présentation du Projet

1.1 Contexte

Ce projet s'inscrit dans le cadre d'un stage au département EEA (Électronique, Énergie Électrique et Automatique) de l'Université de Toulouse. L'objectif est de concevoir et réaliser un système de surveillance connectée de ruches (rucher connecté), intégrant des capteurs physiques, un microcontrôleur STM32, une communication LoRaWAN, et une infrastructure cloud complète pour la visualisation des données. Ce projet a aussi une finalité pédagogique. En effet, ce travail sera réutilisé dans le cadre de TP bureau d'étude sur LoRa. La figure suivante représente un modèle de ruche utilisé durant les travaux pratiques. Toute fois, le système réalisé est applicable à une vraie ruche.

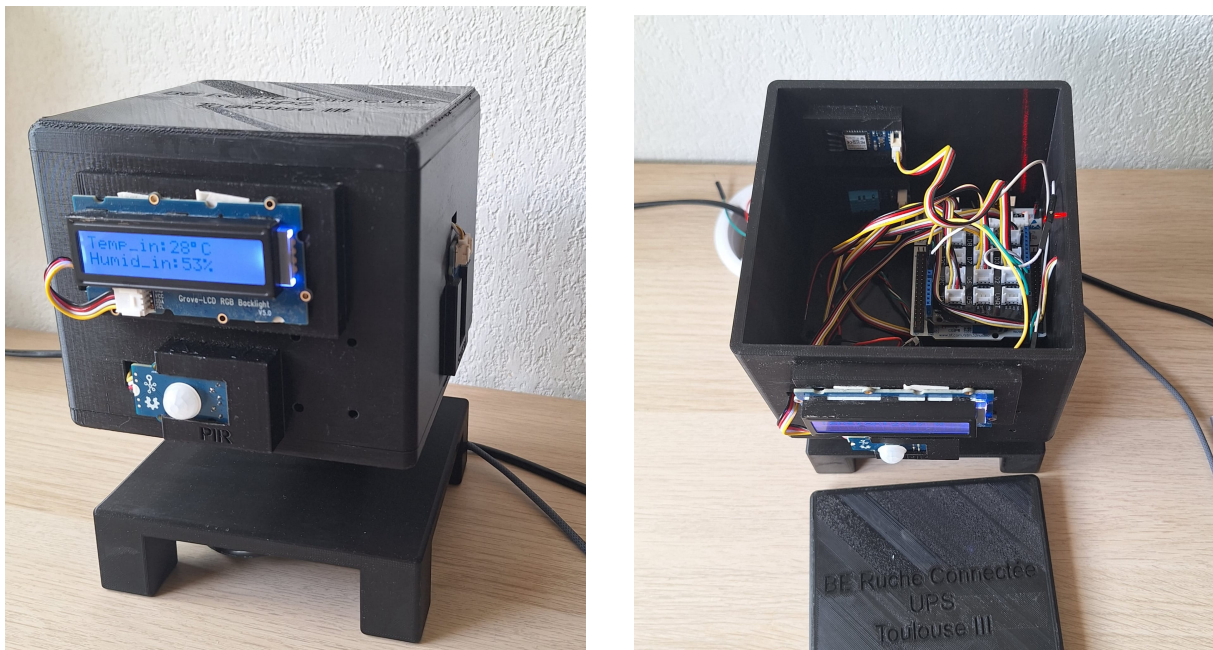


FIGURE 1.1 – Prototype d'une ruche connectée

1.2 Objectifs du système et spécification

Le but du système est de permettre la surveillance d'un rucher. Pour cela, le système doit :

- Mesurer en continu les paramètres physiques de la ruche : température(intérieur et extérieur), humidité(intérieur et extérieur), masse nette, détection présence (PIR)
- Transmettre les données via LoRaWAN (protocole OTAA, bande EU868)

- Agréger et stocker les données sur un serveur LoRaWAN (TTN V3)
- Visualiser en temps réel via un tableau de bord Grafana
- Système autonome, extensible, et documenté pour une réutilisation académique

1.3 Architecture globale

L'architecture du système est la suivante :

Chaîne de traitement complète

STM32L476RG $\xrightarrow{\text{USART3 / AT}}$ **Wio-E5** $\xrightarrow{\text{LoRaWAN EU868}}$ **Gateway SenseCAP M2**
 $\xrightarrow{\text{UDP / IP}}$ **TTN V3** $\xrightarrow{\text{MQTT TLS}}$ **Node-RED** $\rightarrow$ **InfluxDB 2.7** $\rightarrow$ **Grafana**

Le microcontrôleur STM32 constitue le cerveau du système. Il pilote les différents capteurs pour l'acquisition des données, construit la trame de payload, puis la transmet au module LoRa Wio-E5 via une liaison série USART3 au moyen de commandes AT. Le module Wio-E5 encapsule cette trame dans une communication LoRaWAN et la transmet sur la bande EU868 à destination de la gateway SenseCAP M2, qui assure la réception radio et la démodulation du signal.

La gateway relaie ensuite les données reçues vers The Things Network (TTN V3) via une connexion IP, en utilisant le protocole UDP (Semtech Packet Forwarder). TTN V3 agit comme network server : il authentifie le device, décode le payload grâce au formater JavaScript associé à l'application, puis met à disposition les données décodées via une intégration MQTT sécurisée (TLS).

Ces données sont ensuite consommées par Node-RED, qui s'abonne au broker MQTT de TTN, effectue les transformations nécessaires sur les données reçues, puis les insère dans une base de données time-series InfluxDB 2.7. Cette base permet enfin la visualisation des mesures sous forme de tableaux de bord dans Grafana, offrant un suivi en temps réel et historique des différentes grandeurs mesurées par le système.

2. Développement Firmware STM32

2.1 Matériel utilisé

Le matériel utilisé pour réaliser le système est consigné dans le tableau ci-dessous.

Composant	Détail
Microcontrôleur	STM32L476RG (ARM Cortex-M4, 80 MHz, 1 MB Flash)
Module LoRa	Wio-E5-HF (STM32WLE5JC + SX126x)
Capteur T/H	SHT31 (I2C, précision $\pm 0.3^{\circ}\text{C}$ / $\pm 2\%\text{RH}$), DHT11
Capteur poids	Cellule de charge + HX711 (ADC 24 bits)
Capteur présence	PIR passif infrarouge
Affichage	LCD 16×2 (I2C)
Outil de dev	STM32CubeIDE, HAL

TABLE 2.1 – Matériel du nœud capteur (ruche)

2.1.1 Description technique des modules

STM32L476RG (microcontrôleur)

Microcontrôleur ARM Cortex-M4 32 bits de la gamme STM32L4 (ultra-low-power), cadencé jusqu'à 80 MHz, avec unité de calcul flottant (FPU) et jeu d'instructions DSP.

- Mémoire Flash : 1 Mo — RAM : 128 Ko
- Tension d'alimentation : 1,71 – 3,6 V
- Périphériques utilisés : GPIO, I2C1, USART2/USART3, ADC1, TIM16, EXTI
- Rôle dans le système : orchestration de l'acquisition capteurs, mise en forme du payload, pilotage du module LoRa via commandes AT

Wio-E5-HF (module LoRaWAN)

Module radio basé sur le SoC STM32WLE5JC (cœur Cortex-M4 + transceiver LoRa SX126x intégré), piloté par firmware AT propriétaire Seeed.

- Puissance d'émission : 10 à 22 dBm (TXOP jusqu'à 22 dBm @868 MHz)
- Sensibilité de réception : -116,5 dBm à -136,5 dBm (SF12, BW 125 kHz)
- Bandes supportées : EU868 / US915 / AU915 / AS923 / KR920 / IN865
- Consommation : ≈ 26 mA en émission à 14 dBm ; 2,1 μA en veille (mode WOR)
- Interface : UART (AT commands), 3 USART disponibles sur le module
- Protocole : LoRaWAN Class A/B/C, activation OTAA

SHT31 (capteur température/humidité extérieures)

Capteur numérique I2C de Sensirion, basé sur une puce CMOSens combinant élément sensible et électronique de traitement.

- Précision : $\pm 0,3^{\circ}\text{C}$ (température), $\pm 2\%\text{RH}$ (humidité)
- Plage de mesure : -40 à 125°C / 0 à $100\%\text{RH}$
- Interface : I2C, fréquence jusqu'à 1 MHz , adresse 7 bits configurable
- Alimentation : $2,4 - 5,5\text{ V}$

DHT11 (capteur température/humidité intérieures)

Capteur numérique bas coût à protocole propriétaire 1 fil, intégrant un thermistor NTC et un capteur d'humidité résistif.

- Précision : $\pm 2^{\circ}\text{C}$ (température), $\pm 5\%\text{RH}$ (humidité)
- Plage de mesure : 0 à 50°C / 20 à $90\%\text{RH}$
- Fréquence d'échantillonnage : $\approx 1\text{ Hz}$ (une lecture par seconde maximum)
- Interface : protocole 1 fil propriétaire (trame de 5 octets avec checksum), timing géré logiciellement sur PB10

HX711 (interface cellule de charge)

Convertisseur analogique-numérique 24 bits dédié aux capteurs de pesée (cellules de charge en pont de Wheatstone), à interface série bit-bang.

- Résolution : 24 bits
- Gain programmable : 32, 64 ou 128 (canaux A/B)
- Fréquence d'acquisition : 10 ou 80 échantillons/s selon configuration
- Interface : 2 fils (SCK horloge généré par le STM32 sur PA8, DT données lues sur PA9)

Capteur PIR (détection de mouvement)

Détecteur passif infrarouge délivrant une sortie numérique binaire (HAUT/BAS) en fonction de la présence de rayonnement thermique en mouvement dans son champ de détection.

- Sortie : numérique TOR (Tout Ou Rien)
- Alimentation : $3 - 5\text{ V}$
- Interface : GPIO avec interruption externe (EXTI5 sur PB5, front montant)

LCD RGB Backlight (afficheur local)

Afficheur alphanumérique 16×2 caractères, compatible HD44780, avec rétroéclairage RGB piloté par un contrôleur dédié (PCA9633) sur le même bus I2C.

- Résolution : 16 caractères $\times$ 2 lignes
- Interface : I2C (partagée avec le SHT31 sur I2C1)
- Rôle : affichage local en temps réel des mesures (température, humidité, poids, tension batterie, état PIR)

2.2 Schéma de câblage

Le schéma de câblage du système est représenté à la figure 2.1.

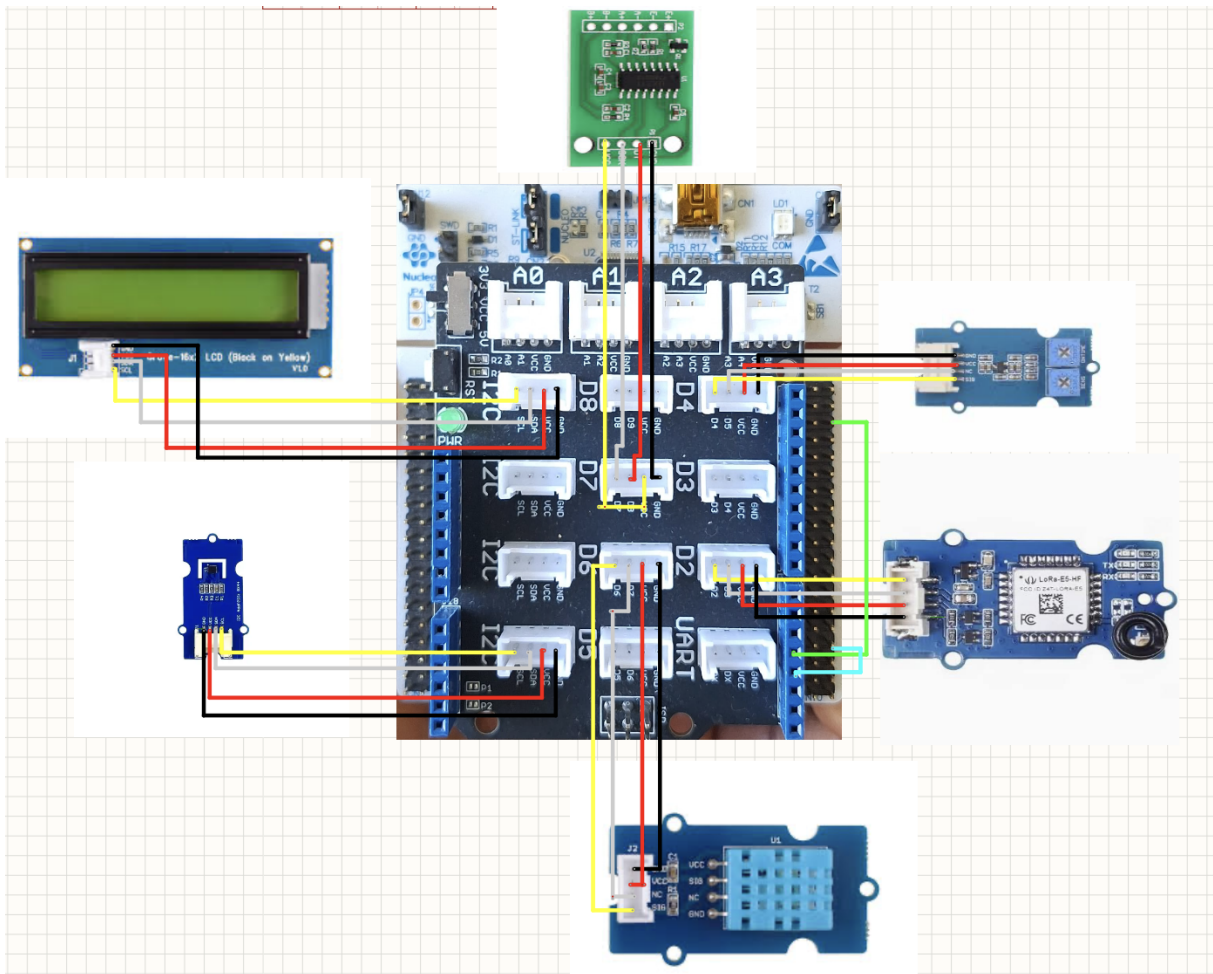


FIGURE 2.1 – Schéma de câblage du système

Le tableau suivant synthétise les connexions entre les modules Grove, le Grove Base Shield et les périphériques du microcontrôleur STM32L476RG.

Module	Grove Base Shield	Périphérique STM32	NVIC	Description
LCD RGB Backlight	I2C	I2C1 (PB8 : SCL, PB9 : SDA)	Désactivé	Affichage local des informations
SHT31	I2C	I2C1 (PB8 : SCL, PB9 : SDA)	Désactivé	Capteur de température/humidité extérieures
DHT11	D6	GPIOB (PB10, entrée)	Désactivé	Capteur de température intérieure
LoRa-E5-HF	D2 (relié à PC4/PC5 par deux câbles, cf. schéma)	USART3 (PC4 : TX, PC5 : RX)	Activé	Émission des données
PIR	D4	GPIOB (PB5, EXTI5)	Activé	Détecteur de mouvement
HX711	D7	GPIOA (PA8 : SCK, PA9 : DT)	Désactivé	Capteur de pesée (cellule de charge)

TABLE 2.2 – Câblage des modules Grove sur le STM32L476RG

Remarque

Le SHT31 et l'écran LCD partagent le même bus I2C1, ce qui impose une gestion séquentielle des accès afin d'éviter les conflits de communication.

2.3 Configuration du fichier .ioc (STM32CubeIDE / CubeMX)

La configuration des périphériques a été réalisée via l'outil graphique de STM32CubeIDE. La figure suivante représente la vue de l'ensemble des périphériques configurés.

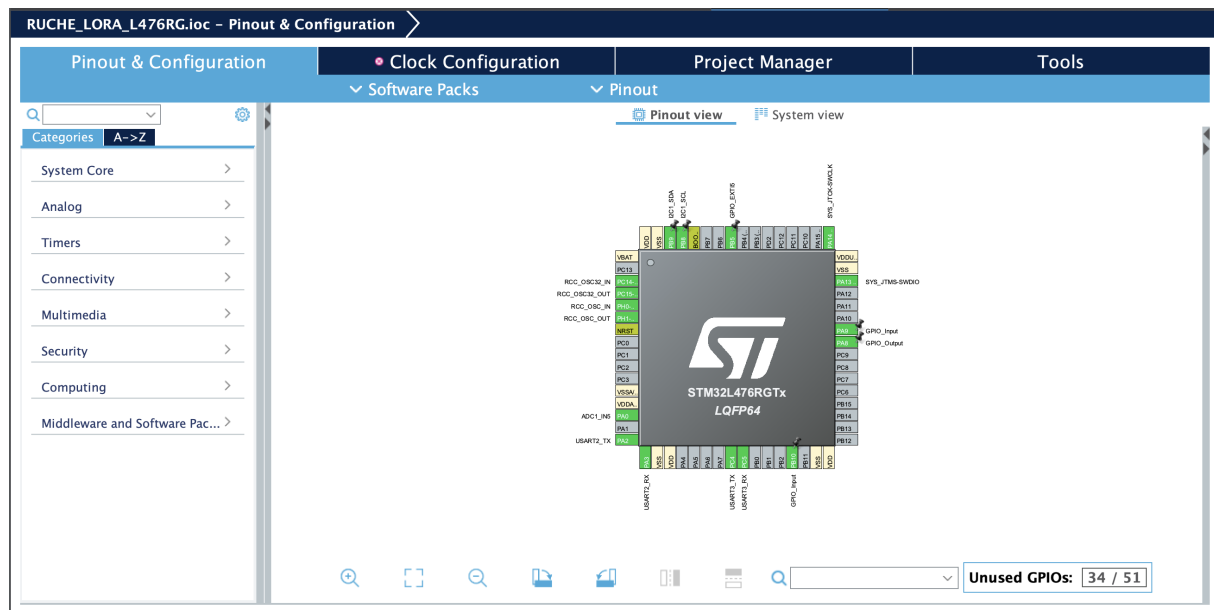


FIGURE 2.2 – Vue d’ensemble de la configuration .ioc

Le tableau ci-dessous détaille les paramètres retenus pour chaque interface.

Connectivité / GPIO	Paramètre	Description
I2C1	Pull-up interne, vitesse Very High	Communication avec le SHT31 et le LCD
USART2	115200 bits/s	Débogage, affichage console PC
USART3	9600 bits/s, NVIC activé	Communication AT entre le STM32 et le module LoRa
PB5	Interruption externe, front montant (EXTI5)	Détection des événements du capteur PIR
PB10	Entrée, sans pull-up ni pull-down	Entrée du signal SIG du DHT11
PA8	Sortie push-pull	Horloge (SCK) pour le HX711
PA9	Entrée, sans pull-up ni pull-down	Entrée des données (DT) du HX711

TABLE 2.3 – Configuration des périphériques dans le fichier .ioc

2.4 Architecture logicielle du firmware

Afin de faciliter la maintenance et la réutilisation du programme, une programmation modulaire a été adoptée : chaque périphérique dispose de sa propre bibliothèque, exposant une interface claire (initialisation, lecture, configuration) et masquant les détails d’implémentation bas niveau. Les bibliothèques suivantes ont été développées ou intégrées :

- `sht31.h` : pilote du capteur de température/humidité SHT31 (I2C)
- `dht11.h` : pilote du capteur DHT11 (protocole 1-fil, timing logiciel)
- `hx711.h` : pilote de la cellule de charge HX711 (lecture bit-bang SCK/DT)
- `lora.h` : gestion des commandes AT du module Wio-E5 sur USART3
- `lib_lcd.h` : pilote de l'écran LCD I2C (bibliothèque fournie)

L'orchestration de ces bibliothèques est assurée par `main.c`, dont le fonctionnement est décrit en détail ci-dessous.

2.4.1 Description du main

Le programme principal suit une architecture **séquentielle et bloquante**, structurée en trois phases : initialisation des périphériques, initialisation applicative (LCD, LoRaWAN, HX711, SHT31), puis boucle infinie d'acquisition et de transmission. La seule exception à ce fonctionnement séquentiel concerne la détection de mouvement, gérée par interruption externe.

Phase d'initialisation matérielle

Après l'appel à `HAL_Init()` et la configuration de l'horloge système (`SystemClock_Config()`), les pilotes des périphériques matériels sont initialisés séquentiellement : GPIO, I2C1, USART2 (debug), TIM16, USART3 (module LoRa) et ADC1.

Phase d'initialisation applicative.

- **LCD** : initialisation via `lcd_init()`, réglage du rétroéclairage en blanc, puis calibration de l'ADC (`HAL_ADCEX_Calibration_Start`), requise avant toute première conversion fiable.
- **LoRaWAN** : le module Wio-E5 est configuré en mode OTAA via `loraWAN_init()`, avec affichage de l'état de connexion sur le LCD pendant la procédure de *join*.
- **HX711** : les broches SCK (PA8) et DT (PA9) sont associées à la structure `hx711`, puis une tare est effectuée par moyennage de 10 lectures brutes consécutives (`HX711_ReadRaw`), stockée dans `hx711.offset`.
- **SHT31** : une première requête de mesure est envoyée (`sht31_request`) pour amorcer le capteur avant la boucle principale.

Boucle principale.

À chaque itération, le programme exécute séquentiellement, avec des temporisations bloquantes (`HAL_Delay`) pour permettre aux modules d'effectuer une requête complète entre chaque étape :

1. **Tension batterie** : conversion ADC en mode polling (`HAL_ADC_PollForConversion`), conversion en volts ($val/4095 \times 3,3$), affichage LCD.
2. **Masse** : lecture via `HX711_GetWeight()`, conversion en kg, affichage LCD.
3. **Mouvement (PIR)** : affichage conditionnel si `pir_detected` est vrai. Ce flag est entièrement piloté par l'interruption externe (voir ci-dessous) et n'est jamais modifié dans la boucle principale.

4. **Température/humidité extérieures (SHT31)** : lecture bloquante (`sht31_read`) et affichage dédié via `lcd_affichage()`.
5. **Température/humidité intérieures (DHT11)** : lecture du protocole 1-fil en 5 octets (RH1, RH2, TC1, TC2, checksum), avec validation par comparaison `Check == SUM` avant mise à jour des variables `temp_in/hum_in`. La gestion du signe (température négative) est traitée via le bit de poids fort de TC1.
6. **Transmission LoRaWAN** : deux trames distinctes sont envoyées via `loraWAN_send_data()` : une trame agrégeant poids, températures/humidités et état PIR, puis une seconde trame dédiée à la tension batterie.

Le cycle complet (mesures, affichages et transmissions) dure environ 16,5 secondes, cette durée étant dominée par les temporisations d’affichage LCD et les délais entre les deux envois LoRaWAN.

Gestion de l’interruption PIR.

La détection de mouvement repose sur une interruption externe configurée sur PB5 (EXTI5). À chaque déclenchement, `HAL_GPIO_EXTI_Callback()` relit directement l’état logique de la broche et met à jour le flag `pir_detected` en conséquence (1 si niveau haut détecté, 0 sinon), assurant que l’état affiché reflète l’état réel du capteur indépendamment du rythme de la boucle principale.

Listing 2.1 – Callback d’interruption externe du capteur PIR

```
1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     if(GPIO_Pin == GPIO_PIN_5)
4     {
5         if(HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_5) == GPIO_PIN_SET)
6             pir_detected = 1;
7         else
8             pir_detected = 0;
9     }
10 }
```

Note technique

Le fichier contient également un mécanisme de réception AT par interruption (`AT_Send()` / `HAL_UART_RxCpltCallback()` sur USART3), non appelé dans le main actuel.

3. LoRaWAN : Communication Wio-E5

3.1 Interface physique STM32 <—> Wio-E5

Signal	Périphérique	Détail
TX / RX	USART3	9600 baud, 8N1, pas de hardware flow control
RESET	GPIO output	Active bas, délai 2.5s après relâchement

TABLE 3.1 – Interface STM32 — Wio-E5

3.2 Mécanisme de communication AT

Le dialogue avec le module Wio-E5 repose sur deux mécanismes distincts implémentés dans `lora.c` : un envoi/réception bloquant avec délai fixe (utilisé pour les tests manuels), et un envoi avec réception pilotée par interruption, réservé à la séquence de jointure OTAA.

3.2.1 Fonction `AT_Send()`

La fonction `AT_Send()` (définie dans `main.c`) envoie une commande AT sur USART3, puis attend la réponse en s'appuyant sur une réception octet par octet déclenchée par interruption (`HAL_UART_Receive_IT`) :

Listing 3.1 – Principe de `AT_Send()`

```
1 void AT_Send(const char* cmd, uint32_t timeout_ms)
2 {
3     memset(rx_buffer, 0, sizeof(rx_buffer));
4     rx_index = 0;
5     rx_complete = 0;
6
7     HAL_UART_Transmit(&huart3, (uint8_t*)cmd, strlen(cmd), 2000);
8     HAL_UART_Receive_IT(&huart3, &rx_byte, 1);
9
10    uint32_t start = HAL_GetTick();
11    while (!rx_complete && (HAL_GetTick() - start) < timeout_ms);
12 }
```

Le callback `HAL_UART_RxCpltCallback()` accumule les octets reçus dans `rx_buffer` et marque `rx_complete = 1` dès réception d'un '`\n`' ou lorsque le buffer atteint 255 octets. Cette approche évite un blocage total du bus UART en cas de réponse tronquée ou absente (sortie garantie par le `timeout_ms` passé en paramètre).

3.2.2 Séquence de jointure OTAA (implémentation réelle)

La fonction `loraWAN_init()` exécute la séquence suivante, chaque étape utilisant `AT_Send()` avec un timeout adapté à la complexité de la commande :

Listing 3.2 – Sequence de jointure OTAA telle qu'implementee dans `loraWAN_init()`

```

1 HAL_Delay(2000) // Attente du boot du
  module
2 AT_Send("AT", 2000) // Test de
  communication -> "+AT: OK"
3 AT_Send("AT+RESET", 2000) // Reset logiciel du
  module
4 HAL_Delay(1000)
5 AT_Send("AT+ID=DevEui", 1000) // Lecture du DevEUI (
  verification)
6 AT_Send("AT+DR=EU868", 1000) // Selection de la
  region EU868
7 AT_Send("AT+ID=DevEui,\"DEV_EUI\"", 1000) // Fixe le DevEUI
8 AT_Send("AT+ID=AppEui,\"APP_EUI\"", 1000) // Fixe l'AppEUI (
  JoinEUI)
9 AT_Send("AT+KEY=AppKey,\"APP_KEY\"", 1000) // Fixe l'AppKey
10 AT_Send("AT+MODE=LWOTAA", 1000) // Active le mode OTAA
11 AT_Send("AT+JOIN", 2000) // Lance la procedure
  de jointure
12 HAL_Delay(1000) // Attente de
  confirmation du JOIN

```

Les identifiants `DEV_EUI`, `APP_EUI` et `APP_KEY` sont définis en constantes dans `lora.h`.

Choix des timeouts

Les timeouts (1 à 2 secondes selon la commande) ont été calibrés empiriquement : les commandes de simple configuration (`AT+ID`, `AT+DR`, `AT+KEY`) répondent en général en quelques centaines de millisecondes, tandis que `AT+JOIN` nécessite un délai plus long pour laisser le temps à l'échange Join Request / Join Accept de s'effectuer sur l'interface radio.

3.3 Mode test / point-à-point (débugage RF)

En complément du mode LoRaWAN applicatif, `lora.c` propose un ensemble de fonctions dédiées au mode `TEST` du Wio-E5, utilisé pour valider la chaîne radio indépendamment de tout serveur réseau (LoRaWAN), utile en phase de mise au point ou pour des mesures RF ponctuelles.

Listing 3.3 – Configuration radio en mode `TEST` (emetteur)

```

1 AT+MODE=TEST
2 AT+TEST=RFCFG,868,SF7,125,8,8,14,ON,OFF,OFF
3 AT+TEST=TXLRSTR

```

Les paramètres de `RFCFG` sont, dans l'ordre : fréquence (868 MHz), facteur d'étalement SF7, largeur de bande 125 kHz, préambule (8 symboles TX/RX), puissance d'émission (14

dBm, portée à 22 dBm pour LoRa_P2P_Init), et trois indicateurs CRC/IQ inversé/sortie continue désactivés.

Les fonctions `LoRa_emetteur()` et `LoRa_recepteur()` configurent respectivement un nœud en émission (TXLRSTR) ou en réception continue (RXLRPKT) avec la même configuration radio, permettant des essais de liaison point-à-point en laboratoire.

3.4 Réception et décodage des trames descendantes

La fonction `LoRa_afficher_rx()` extrait et décode les trames reçues en mode test. La réponse AT du module encode les données reçues en hexadécimal ASCII (motif `+TEST:RX "..."`); la fonction utilitaire `hex_to_ascii()` convertit cette chaîne hexadécimale en texte lisible, affiché à la fois sur l'UART de debug et sur l'écran LCD (ligne 1 : données brutes hexadécimales, ligne 2 : données décodées ASCII).

3.5 Transmission des données applicatives

La transmission des mesures capteurs vers le réseau LoRaWAN est assurée par `loraWAN_send_data()`, appelée deux fois par cycle dans la boucle principale (une trame de mesures, une trame de tension batterie — cf. section 2.4.1).

Listing 3.4 – Envoi d'une trame applicative

```
1 void loraWAN_send_data(char* data)
2 {
3     uint8_t cmd[254];
4     memset(rxBuf, 0, sizeof(rxBuf));
5     snprintf((char*)cmd, sizeof(cmd), "AT+MSG=\"%s\"\\r\\n", data);
6     HAL_UART_Transmit(&huart3, cmd, strlen((char*)cmd), 2000);
7     HAL_Delay(200);
8     HAL_UART_Receive(&huart3, rxBuf, sizeof(rxBuf), 5000);
9     HAL_Delay(200);
10    HAL_UART_Transmit(&huart2, rxBuf, strlen((char*)rxBuf), 5000);
11    HAL_Delay(5000);
12 }
```

Le payload est actuellement transmis en **texte ASCII brut** via la commande `AT+MSG` (et non en binaire compact via `AT+MSGHEX`). Ce choix simplifie le débogage (lecture directe des trames dans la console TTN) mais consomme davantage d'octets utiles, un point à surveiller vis-à-vis des limites de taille de payload imposées par le *Data Rate* EU868 utilisé. Le passage à un encodage binaire compact avec décodeur JavaScript associé constitue une piste d'optimisation identifiée mais non encore implémentée.

3.6 Étude des caractéristiques de la modulation LoRa

Avant de fixer les paramètres radio du système, une étude des caractéristiques de la couche physique LoRa (Chirp Spread Spectrum) a été menée, afin de comprendre l'influence de chaque paramètre sur le compromis portée / débit / consommation du canal, et d'orienter

les choix de configuration en conséquence.

3.6.1 Principe de la modulation LoRa

LoRa repose sur une modulation à étalement de spectre par chirps (*Chirp Spread Spectrum*, CSS), un procédé emprunté à la technologie RADAR. Pendant la durée de transmission de chaque symbole, le modulateur élabore une onde sinusoïdale dont la fréquence instantanée varie linéairement autour d'une fréquence porteuse f_c . On distingue :

- les **upchirps**, pour lesquels la fréquence croît linéairement au cours du temps ;
- les **downchirps**, pour lesquels la fréquence décroît linéairement.

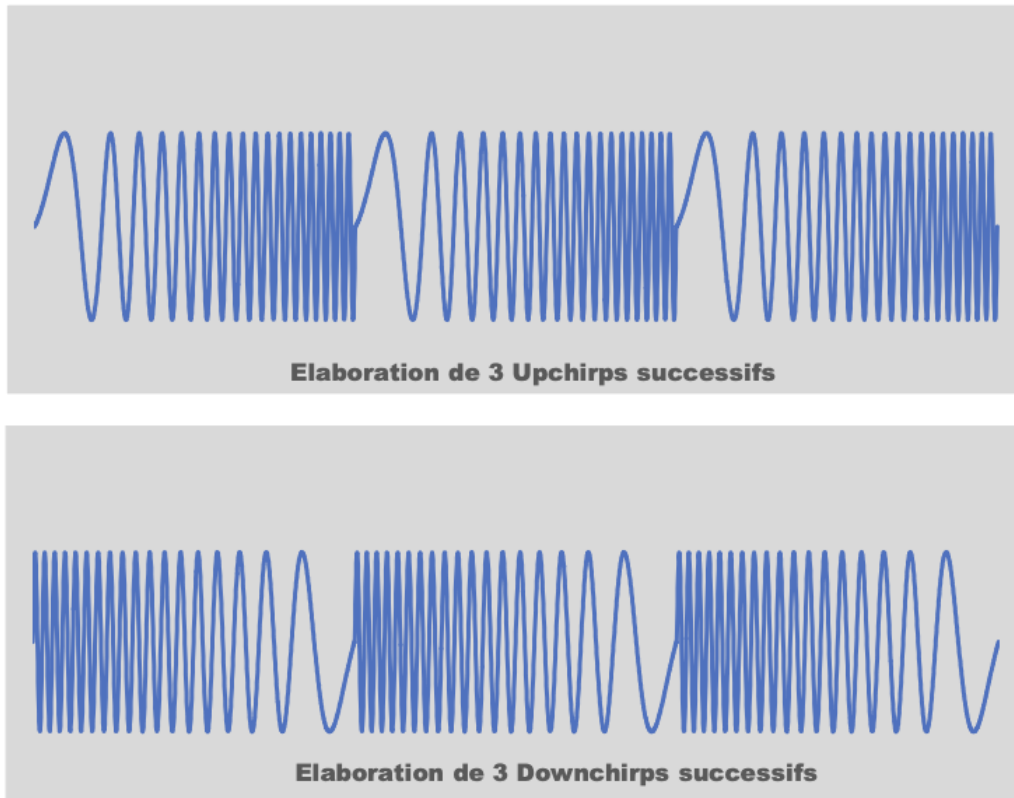


FIGURE 3.1 – Élaboration d'upchirps et de downchirps successifs [1]

La modulation LoRa utilise exclusivement des upchirps pour coder les données. Le codage de l'information consiste, à l'instant de transmission d'un nouveau symbole n , à faire démarrer le chirp à une fréquence de départ f_x proportionnelle à la valeur du symbole : plus n est élevé, plus le chirp débute haut dans la bande, puis "boucle" en fin de bande vers le bas pour compléter son excursion sur la durée T_s du symbole (discontinuité de fréquence caractéristique visible sur un spectrogramme). Le décalage temporel de cette discontinuité au sein du symbole encode ainsi directement la valeur transmise.

Cette modulation confère à LoRa une forte robustesse au bruit et aux interférences à bande étroite, au prix d'un débit utile réduit par rapport à des modulations classiques (FSK, OOK) à puissance égale.

Trois paramètres PHY gouvernent conjointement ce compromis : le facteur d'étalement (SF), la bande passante (BW) et le taux de codage correcteur d'erreur (CR).

3.6.2 Étude du Spreading Factor (SF)

Le SF (7 à 12 en LoRa) fixe le nombre de chirps par symbole (2^{SF} chirps/symbole) et détermine directement le compromis sensibilité/débit :

- **SF faible (SF7)** : débit élevé, temps d'antenne (*Time on Air*(ToA)) court, sensibilité de réception plus faible (≈ -123 dBm à BW125).
- **SF élevé (SF12)** : débit faible, ToA long, sensibilité maximale (≈ -137 dBm à BW125), adaptée aux liaisons longue portée ou en environnement dégradé.
- Règle générale observée : chaque incrément de SF apporte $\approx 2,5$ dB de gain de sensibilité, mais double approximativement le temps d'antenne à débit symbole égal.

SF	Débit approx. (BW125)	Sensibilité approx.	Usage typique
7	5470 bit/s	-123 dBm	Courte portée, débit élevé
9	1760 bit/s	-129 dBm	Portée moyenne
12	250 bit/s	-137 dBm	Longue portée, faible débit

TABLE 3.2 – Compromis débit/sensibilité selon le Spreading Factor (BW = 125 kHz), issu de l'étude comparative

3.6.3 Étude de la bande passante (BW)

La bande passante (125 / 250 / 500 kHz en EU868) influe sur le débit symbole et le bruit thermique intégré par le récepteur :

- BW125 kHz : sensibilité maximale, débit le plus faible — bande standard pour la plupart des DataRates EU868 (DR0 à DR5).
- BW250 kHz : réservée au DataRate DR6, débit supérieur au prix d'une sensibilité réduite d'environ 3 dB.

L'étude confirme que la bande 125 kHz constitue le choix par défaut le plus robuste, cohérent avec l'usage majoritaire observé sur les réseaux EU868 (TTN inclus).

3.6.4 Étude du Coding Rate (CR)

Le CR (4/5 à 4/8) ajoute de la redondance via un code de Hamming pour la correction d'erreur en avant (FEC). Un CR plus élevé (ex. 4/8) améliore la robustesse au bruit impulsionnel au prix d'un débit utile réduit. En mode LoRaWAN (OTAA), le CR effectif est géré automatiquement par la pile protocolaire du module et n'est pas configuré manuellement — contrairement au mode TEST du Wio-E5, où l'ensemble des paramètres radio (SF, BW, préambule, puissance) est fixé explicitement via la commande `AT+TEST=RFCFG`.

3.6.5 Étude de la puissance d'émission et cadre réglementaire

- **14 dBm (25 mW)** : conforme à la limite ETSI EN 300 220 pour la plupart des sous-bandes EU868 avec *duty cycle* 1%, sans nécessiter d'écoute avant transmission (LBT/AFA).
- **22 dBm** : niveau maximal testé en mode point-à-point pour visualiser le signal avec l'analyseur de spectre.

Contrainte de *duty cycle* EU868

La réglementation ETSI impose un rapport cyclique maximal d'utilisation du canal radio, qui limite le nombre et la durée des trames émissibles par heure :

Sous-bande EU868	Duty cycle	Remarque
868,0 – 868,6 MHz	1%	Sous-bande principale, utilisée par TTN par défaut
869,4 – 869,65 MHz	10%	Puissance max. 27 dBm, canal RX2 par défaut TTN

TABLE 3.3 – Contraintes de duty cycle par sous-bande EU868

3.6.6 Débit binaire utile et impact conjoint de SF, BW et CR

Au-delà de la durée de symbole $T_s = 2^{SF}/BW$, le débit binaire utile réel R_b dépend conjointement des trois paramètres SF, BW et CR selon :

$$R_b = SF \cdot \frac{BW}{2^{SF}} \cdot CR \quad (\text{en bits/s})$$

et la vitesse de modulation (symbol rate) associée :

$$R_s = \frac{BW}{2^{SF}} \cdot CR \quad (\text{en Bauds})$$

Le tableau suivant donne les débits utiles réels (en bits/s) pour un taux de codage $CR = 4/5$, selon les valeurs de SF et BW autorisées par la norme en Europe :

SF	BW (kHz)	R_b (CR=4/5)	Portée relative
7	250	10 938 bit/s	la plus faible
7	125	5 469 bit/s	
8	125	3 125 bit/s	
9	125	1 758 bit/s	
10	125	977 bit/s	
11	125	537 bit/s	
12	125	293 bit/s	la plus élevée

TABLE 3.4 – Débit binaire utile selon SF et BW, CR=4/5 (norme LoRaWAN Europe). Valeurs issues de la documentation constructeur Semtech.

Selon la norme, 4 valeurs de CR sont possibles (4/5, 4/6, 4/7, 4/8) ; combinées aux 6 valeurs de SF (7 à 12) et aux 2 valeurs de BW utilisées en Europe (125/250 kHz), on obtient au total 28 débits utiles distincts, compris entre 180 bit/s et 11 kbit/s.

Portée vs débit

À puissance d'émission constante, le constructeur Semtech indique que la portée croît avec le facteur d'étalement SF : en environnement dégagé (*Line Of Sight*), la portée peut atteindre jusqu'à 14 km à SF12, contre une portée bien plus réduite à SF7, cohérent avec le compromis débit/sensibilité étudié section précédente.

3.6.7 Étude du mécanisme d'adaptation dynamique (ADR)

Contrairement au mode TEST où le SF est fixé manuellement (RFCFG,868,SF7,...), le mode LWOTAA utilisé en production laisse la main au *network server* TTN via l'ADR (Adaptive Data Rate) : celui-ci ajuste dynamiquement le DataRate (couple SF/BW) et la puissance d'émission du device en fonction de la qualité de liaison observée (RSSI/SNR des trames reçues par la gateway), afin d'optimiser conjointement portée, consommation et charge du canal. Cette caractéristique a été identifiée comme un argument en faveur d'une configuration initiale « prudente » côté device, l'ADR se chargeant ensuite d'affiner les paramètres en fonction des conditions réelles du terrain.[3]

3.7 Synthèse de l'étude et paramètres retenus

L'étude menée ci-dessus a orienté les choix de configuration suivants pour le système :

Paramètre	Valeur retenue	Justification issue de l'étude
Spreading Factor	SF7	Distance gateway-device réduite lors des essais ; minimise le ToA, donc l'impact sur le duty cycle
Bande passante	125 kHz	Compromis standard, sensibilité maximale à débit acceptable
Puissance d'émission	14 dBm (essais applicatifs) / 22 dBm (tests de portée P2P)	Conformité ETSI en sous-bande principale ; 22 dBm réservé aux essais ponctuels
Bande / Région	EU868 (AT+DR=EU868)	Réglementation ETSI applicable en France/UE
Mode d'activation	OTAA (AT+MODE=LWOTAA)	Pas de provisioning statique des clés de session ; plus sûr que ABP
Format payload	ASCII (AT+MSG)	Simplicité de débogage ; optimisation binaire identifiée comme piste d'amélioration
Timeout AT+JOIN	2000 ms	Laisser le temps à l'échange radio Join Request/Accept
Délai post-reset	2000 ms (boot) + 1000 ms (post-RESET)	Délai minimal requis par le Wio-E5 avant d'accepter des commandes AT
Délai inter-frames	5000 ms (HAL_Delay entre les deux trames par cycle)	Marge de sécurité vis-à-vis du duty cycle, en tenant compte des trames MAC générées par la pile LoRaWAN

TABLE 3.5 – Paramètres LoRaWAN retenus à l'issue de l'étude

4. Gateway LoRaWAN(SenseCAP M2)

4.1 Présentation du matériel

Paramètre	Valeur
Modèle	Seeed SenseCAP M2 LoRaWAN Indoor Gateway
Gateway EUI	2C :F7 :F1 :10 :60 :30 :00 :1B
Firmware	OpenWrt 21.02.0 r1-20220615 r16279-5cc0535800
Interface admin	LuCI (web, port 80)
Bande	EU868 (862–870 MHz)
Mode	Packet Forwarder

TABLE 4.1 – Identification gateway SenseCAP M2

Un conflit `gateway_eui_taken` a été rencontré lors de l'enregistrement sur TTN : la gateway était déjà revendiquée par un autre compte (précédent stagiaire). Résolution : modification de l'EUI à partir du serveur de configuration de la gateway.

Paramètre	Valeur configurée
Serveur TTN	<code>eu1.cloud.thethings.network</code>
Port UDP uplink	1700
Port UDP downlink	1700
Protocole	Semtech UDP Packet Forwarder

TABLE 4.2 – Paramètres Packet Forwarder

5. The Things Network (TTN V3)

5.1 Configuration du compte et de l'application

Paramètre	Valeur
Serveur	The Things Stack V3 — Cluster eu1
Console	<code>eu1.cloud.thethings.network</code>
Application ID	<code>rucher-ut</code>
Région	Europe EU868
Plan fréquences	EU_863_870
Mode activation	OTAA

TABLE 5.1 – Paramètres TTN V3

5.2 Procédure OTAA

La procédure OTAA (Over-The-Air Activation) se déroule en deux phases :

1. **Join Request** : le device envoie JoinEUI + DevEUI + AppKey (chiffré)
2. **Join Accept** : TTN répond avec DevAddr, NwkSKey, AppSKey
3. Le device peut ensuite envoyer des trames de données chiffrées

5.3 Payload Formatter (décodeur TTN)

Un décodeur JavaScript est configuré sur TTN pour convertir les bytes bruts en valeurs lisibles. La fonction de décodage est représentée à l'annexe **A.1**.

Cohérence avec le format de payload actuel

Ce formatter binaire correspond au format **cible** identifié comme piste d'optimisation (cf. chapitre 3). Le payload actuellement transmis par le device étant en ASCII brut (**AT+MSG**), ce décodeur devra être adapté (ou un décodeur ASCII simple mis en place côté TTN) tant que la bascule vers le format binaire compact n'est pas effective.

6. Stack Backend : MQTT, Node-RED, InfluxDB, Grafana

6.1 Architecture déployée

Toute la stack backend est implémenté sur un raspberry pi. Le système devant fonctionner sans interruption, on a jugé nécessaire d'utiliser un raspberry pour réduire la consommation énergétique.

Service	Port	Rôle
Node-RED	1880	Réception MQTT, transformation, écriture InfluxDB
InfluxDB 2.7	8086	Base de données time-series
Grafana	3000	Tableau de bord temps réel

TABLE 6.1 – Services Docker déployés

6.2 Connexion MQTT TTN → Node-RED

Paramètre	Valeur
Broker MQTT	<code>eu1.cloud.thethings.network</code>
Port	8883 (TLS)
Username	<code>rucher-ut@ttn</code>
Password	API Key TTN
Topic souscrit	<code>v3/ttig-ruecher-ut@ttn/devices/ruche1-ut/up</code>

TABLE 6.2 – Paramètres MQTT

6.3 Pipeline Node-RED

Le flux Node-RED traite les messages MQTT TTN et les écrit dans InfluxDB :

1. **MQTT In** : souscription au topic TTN, réception JSON
2. **Function** : extraction des champs `decoded_payload` (température, humidité, poids, PIR). La fonction de décodage du payload est représenté à l'annexe **A.1**.
3. **InfluxDB Out** : écriture dans le bucket `rucher` avec tags `device_id`

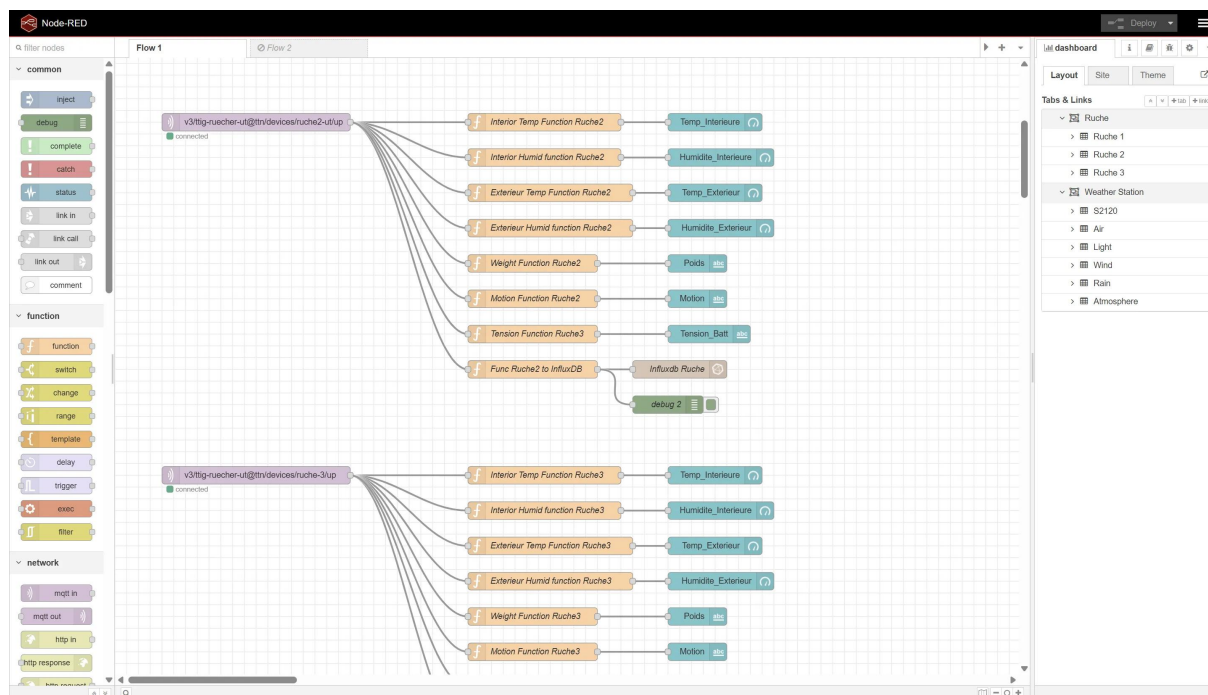


FIGURE 6.1 – Configuration du flux sur Node-RED

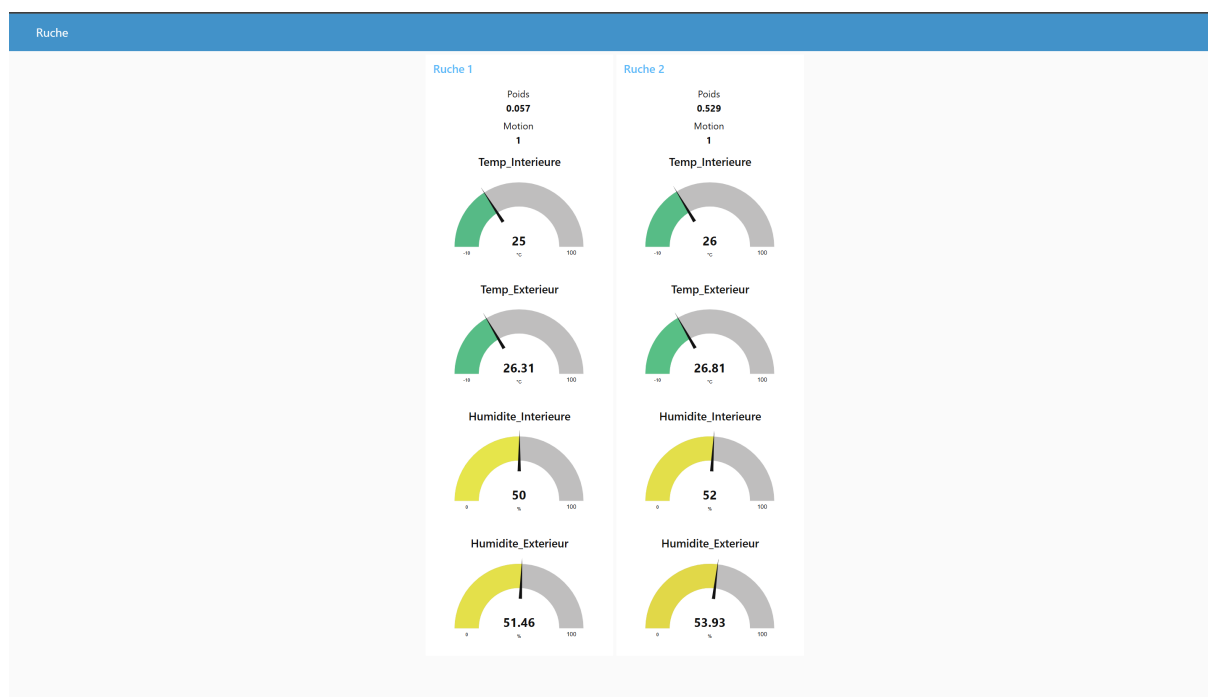


FIGURE 6.2 – Visualisation des informations du rucher avec Node-RED

6.4 InfluxDB 2.7

InfluxDB permet de récupérer les données de Node-RED. Ces dernières sont exportées vers Grafana pour afficher les courbes d'évolutions.

— Organisation : **rucher-org**

- Bucket : **rucher**
- Rétention : 30 jours
- Authentification : Token API

6.5 Grafana

Le tableau de bord Grafana interroge InfluxDB via Flux et affiche :

- Courbe température en temps réel
- Courbe humidité
- Jauge poids de la ruche
- Indicateur binaire PIR (présence)
- Compteur de trames reçues (RSSI, SNR)



FIGURE 6.3 – Tableau de bord sur Grafana

7. Prise en main de l'analyseur FPL - Étude de la couche physique LoRa

7.1 Contexte

En complément du développement système, une phase de familiarisation avec un analyseur de spectre Rohde & Schwarz (FLP spectrum analyser 5KHz - 3GHz) a été menée, dans l'objectif d'observer directement les caractéristiques physiques du signal LoRa émis par le module Wio-E5 (chirps, spectre occupé, évolution temps-fréquence), et de faire le lien entre les paramètres radio configurés (SF, BW, puissance) et leur traduction concrète sur le signal réel.

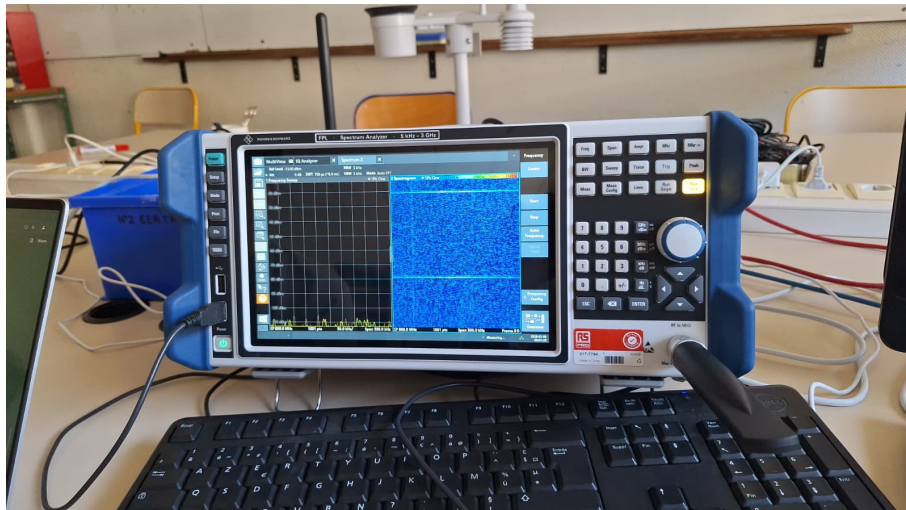


FIGURE 7.1 – FLP spectrum analyser 5KHz - 3GHz

Périmètre du travail réalisé

Ce travail s'est limité à la **prise en main de l'instrument** et à l'**observation qualitative** du signal LoRa (visualisation des chirps, du spectre, du spectrogramme). **Aucun décodage logiciel** des trames capturées (dechirp, FFT, extraction de bits) n'a été implémenté à ce stade, cet aspect est identifié comme perspective pour la suite du projet (cf. chapitre 8).

7.2 Choix des paramètres de configuration de l'analyseur

La configuration de l'analyseur (fréquence centrale, span, RBW/VBW, temps d'acquisition, déclenchement) n'est pas arbitraire : chaque réglage découle directement des caractéristiques du signal LoRa que l'on cherche à observer (bande occupée, durée du symbole,

durée totale de la trame). Cette section détaille le raisonnement suivi pour chaque paramètre.

7.2.1 Fréquence centrale et span

Le signal étant émis en test radio à 868 MHz avec une bande passante configurée de 125 kHz (cf. chapitre 3), la fréquence centrale de l'analyseur a été réglée sur 868 MHz, avec un **span** couvrant une marge autour de la bande occupée réelle :

- Span minimal théorique : 125 kHz (largeur du chirp, du bas au haut de la bande)
- Span retenu en pratique : ≈ 2 à $3 \times$ la bande occupée (250–400 kHz), afin de visualiser clairement les bords du chirp et de détecter d'éventuels débordements spectraux (fuites hors bande, harmoniques proches) sans pour autant diluer la résolution sur la zone utile.

7.2.2 Résolution en fréquence (RBW) et compromis temps-fréquence

La *Resolution Bandwidth* (RBW) détermine la finesse avec laquelle l'analyseur distingue deux composantes fréquentielles proches, mais elle est directement liée par le principe d'incertitude temps-fréquence au temps d'observation nécessaire pour chaque point de mesure :

- **RBW trop fine** : bonne résolution fréquentielle, mais temps d'intégration long → risque de "flouter" le chirp, dont la fréquence instantanée varie rapidement dans le temps (le balayage complet de 125 kHz s'effectue en quelques dizaines à quelques centaines de microsecondes selon le SF).
- **RBW trop large** : bonne résolution temporelle (on suit bien l'évolution rapide du chirp), mais résolution fréquentielle dégradée et niveau de bruit affiché plus élevé (le bruit affiché croît avec $\sqrt{RBW}$).

Le compromis retenu privilégie une RBW suffisamment large pour suivre la pente du chirp sur le spectrogramme (de l'ordre du kHz à quelques dizaines de kHz selon le mode d'affichage utilisé), quitte à sacrifier une résolution fréquentielle fine, l'objectif étant ici d'observer la **forme** du balayage plutôt que de mesurer précisément un niveau de puissance à une fréquence donnée.

7.2.3 Temps d'acquisition

Le temps d'acquisition (durée de capture / durée de balayage) a été dimensionné à partir de la durée réelle du signal à observer, elle-même déduite des paramètres radio configurés :

- **Durée d'un symbole** : $T_{sym} = 2^{SF}/BW$, soit environ 1,02 ms pour SF7/BW125 kHz.
- **Durée du préambule** : plusieurs symboles consécutifs (chirps montants identiques), typiquement 8 symboles dans la configuration retenue (RFCFG, ..., 8, 8, ...), soit ≈ 8 ms.
- **Durée totale de la trame (*Time on Air*)** : préambule + en-tête + payload, de l'ordre de quelques dizaines de millisecondes pour une trame de test courte en SF7.

Le temps d'acquisition a donc été réglé avec une marge confortable au-delà de la durée de trame estimée (typiquement 1,5 à 2× la durée théorique), afin de garantir la capture complète du préambule jusqu'à la fin du payload, même en cas de léger décalage de déclenchement.

7.2.4 Déclenchement (trigger)

Le signal LoRa étant émis de façon brève et ponctuelle (quelques dizaines de ms) au sein d'une longue période d'inactivité radio, une acquisition en mode libre (*free run*) est inefficace pour capturer la trame de façon fiable. Un déclenchement sur seuil de puissance (*power trigger*) a donc été utilisé, réglé légèrement au-dessus du niveau de bruit ambiant, afin que l'analyseur ne démarre l'acquisition qu'au moment effectif de l'émission.

7.2.5 Niveau de référence et atténuation

Le niveau de référence (*reference level*) et l'atténuation d'entrée ont été ajustés en fonction de la puissance d'émission attendue (14 à 22 dBm en sortie du Wio-E5, cf. chapitre 3), corrigée des pertes de câblage/connectique entre l'émetteur et l'analyseur, afin de :

- Éviter la saturation de l'étage d'entrée (compression, distorsion du chirp observé)
- Conserver une dynamique d'affichage suffisante pour distinguer le signal du plancher de bruit

7.2.6 Fréquence d'échantillonnage (mode IQ / capture transitoire)

Pour les captures en mode IQ (bande de base), la fréquence d'échantillonnage a été choisie pour respecter le critère de Nyquist vis-à-vis du span observé, avec une marge de suréchantillonnage :

- Critère minimal : $F_e \geq \text{span observé}$ (ici $\approx 250\text{--}400$ kHz)
- Marge appliquée en pratique : suréchantillonnage d'un facteur 2 à 4 par rapport au span, pour limiter les effets de repliement en bord de bande et faciliter un éventuel post-traitement ultérieur des échantillons capturés.

7.2.7 Exemple de capture : Mode Transient Analysis (logiciel VSE)

La figure 7.2 illustre une capture réelle effectuée avec le module *Transient Analysis* du logiciel R&S VSE, configuré en modèle *Chirp*, avec les paramètres suivants : fréquence centrale 868,3 MHz, temps de mesure 100 ms, bande de mesure (Meas BW) 750 kHz, fréquence d'échantillonnage (SRate) 937,5 kHz, atténuation 0 dB, déclenchement sur puissance IF (TRG:IFP).

Cette vue combine plusieurs représentations complémentaires du même signal capturé, chacune mettant en évidence un aspect différent du chirp LoRa :

- **Full RF Spectrum (haut gauche)** : représente l'amplitude du signal (en dBm) en fonction de la fréquence, centrée sur 868,3 MHz avec une résolution de 75 kHz/division. On y observe un lobe principal élargi caractéristique de l'étalement de spectre par chirp, dont les bords sont repérés par les marqueurs *AR* à 868,237 MHz et 868,362 MHz. Le marqueur M1 (-90,14 dBm à 868,550 MHz) permet de situer le niveau de bruit hors bande utile.

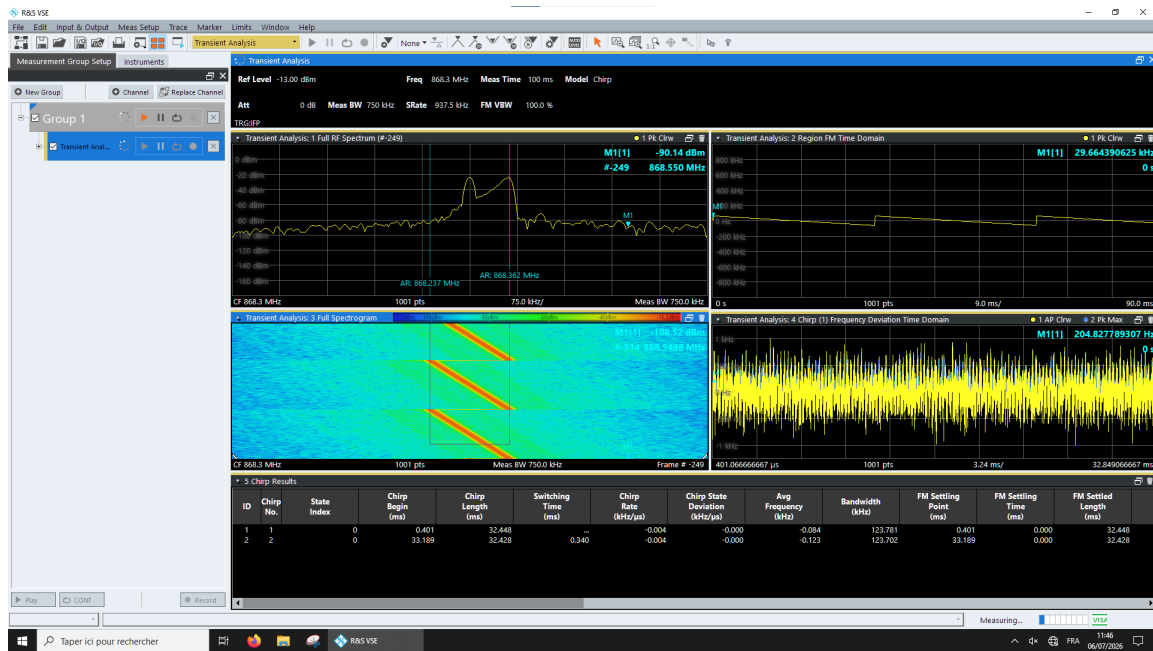


FIGURE 7.2 – Capture d’écran du logiciel R&S VSE — mode Transient Analysis, modèle Chirp

- **Region FM Time Domain (haut droite)** : affiche la déviation de fréquence instantanée (par rapport à la porteuse) en fonction du temps, sur une fenêtre de 90 ms. La forme en paliers ascendants correspond à la variation de fréquence du chirp au cours du temps : chaque palier traduit le passage d’un symbole à un autre, la fréquence instantanée repartant du bas de la bande à chaque nouveau symbole (comportement typique d’une succession de *up-chirps*, comme lors de la transmission du préambule).
- **Full Spectrogram (milieu gauche)** : vue temps-fréquence (*waterfall*) où la couleur code la puissance instantanée du signal. Les bandes diagonales orangées visibles sur le fond bleu (bruit) matérialisent directement la signature du chirp LoRa : un balayage linéaire et continu de la fréquence sur toute la bande passante au cours du temps. La pente de ces diagonales est directement liée au couple (SF, BW) configuré, plus le SF est élevé, plus la pente est faible (balayage plus lent).

Mesures associées

Le tableau de résultats *Chirp Results* généré automatiquement par le logiciel (non reproduit ici) confirme ces observations avec des valeurs mesurées : une bande passante occupée d’environ 123,7–123,8 kHz (cohérente avec les 125 kHz configurés) et une durée de symbole mesurée d’environ 32,4 ms — à comparer à la valeur théorique $T_{sym} = 2^{SF}/BW$, qui donne $\approx 32,77$ ms pour SF12/BW125 kHz. Cette correspondance permet de valider indirectement, par la mesure, les paramètres radio réellement émis par le module.

Synthèse du raisonnement

Chaque paramètre de l'analyseur a été fixé en remontant des caractéristiques connues du signal ($BW = 125$ kHz, $T_{sym} \approx 1$ ms, ToA de quelques dizaines de ms, puissance de 14–22 dBm) vers les réglages instrumentaux correspondants, plutôt qu'en utilisant des valeurs par défaut génériques, démarche nécessaire pour obtenir une observation exploitable d'un signal aussi particulier qu'un chirp LoRa (variation rapide de fréquence, émission brève et intermittente).

7.3 Observations réalisées sur le signal LoRa

L'analyseur a été utilisé pour visualiser et caractériser plusieurs aspects du signal émis en mode TEST par le Wio-E5 (paramètres SF7, BW125 kHz, cf. chapitre 3) :

- **Visualisation des chirps** : observation de la forme caractéristique en dents de scie du chirp LoRa dans le domaine temps-fréquence (balayage linéaire de fréquence sur toute la bande passante à chaque symbole).
- **Spectre occupé** : mesure de la largeur de bande occupée par le signal, permettant de vérifier la cohérence avec le paramètre BW configuré (125 kHz).
- **Spectrogramme / vue *waterfall*** : observation de la succession des symboles (chirps up/down) dans le temps, mettant en évidence le préambule (répétition de chirps montants) précédant la partie utile de la trame.
- **Durée de symbole** : estimation visuelle de la durée d'un symbole LoRa à partir du spectrogramme, cohérente avec la formule théorique $T_{sym} = 2^{SF}/BW$ pour SF7/BW125.
- **Capture IQ** : enregistrement de segments du signal en bande de base (I/Q) autour des instants de transmission, en vue d'une exploitation ultérieure (traitement hors ligne).

Cette phase d'observation a permis de valider empiriquement la cohérence entre la configuration radio définie côté firmware (chapitre 3) et le comportement réel du signal émis, sans toutefois aller jusqu'à la démodulation/décodage des données transportées.

7.3.1 Interprétation de la structure préambule/payload observée

La succession de chirps identiques observée en début de trame sur le spectrogramme (avant la partie porteuse des données) correspond à la structure de trame normalisée par Semtech : chaque trame LoRa démarre par un **préambule**, constitué d'un nombre programmable n d'upchirps "complets" identiques (équivalents à la transmission répétée du symbole 0), suivi de 2 downchirps spécifiques servant de marqueurs de synchronisation. Ce n'est qu'après ces 2 downchirps que débute l'en-tête optionnel puis les données utiles (*payload*).

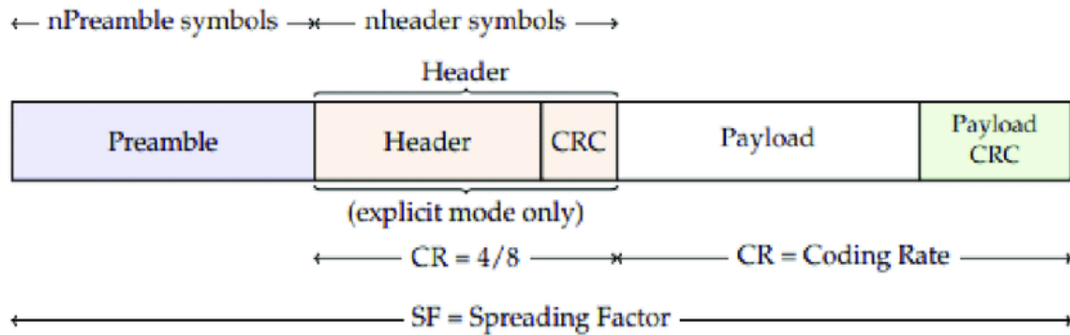


FIGURE 7.3 – Structure temporelle d’une trame LoRa [2]

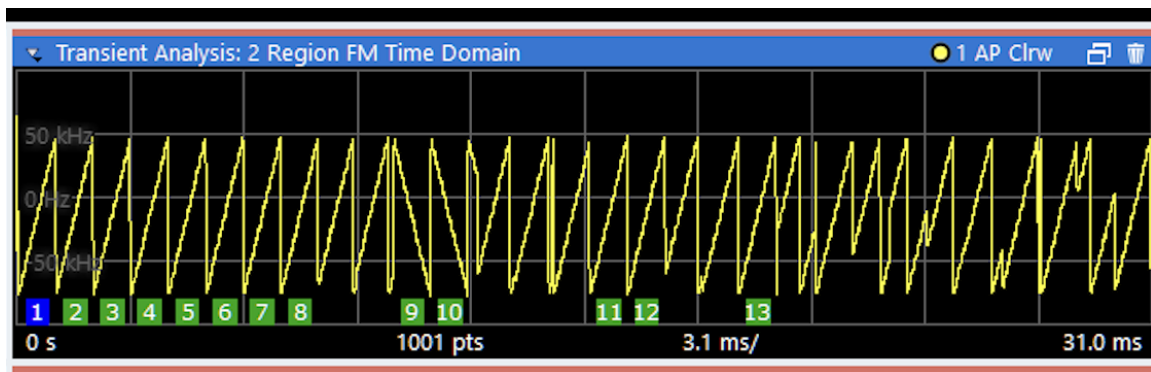


FIGURE 7.4 – Structure temporelle d’une trame LoRa : préambule (upchirps + 2 downchirps) suivi des données

Cette structure explique la présence, sur la vue *Full Spectrogram* de la capture (figure 7.2), de plusieurs diagonales consécutives de pente identique avant la zone où le signal devient porteur d’information : il s’agit du préambule, dont le rôle est précisément de permettre au récepteur de se synchroniser sur le rythme du chirp émis avant de commencer la démodulation effective des données.

8. Bilan et Perspectives

8.1 État d'avancement

Le projet réalisé au cours de ce stage avait pour objectif de concevoir et de mettre en œuvre un système de surveillance à distance d'un rucher reposant sur la technologie LoRaWAN. Le système permet d'acquérir différentes grandeurs physiques au niveau des ruches, de transmettre les données sans fil, puis de les stocker et de les visualiser à distance.

Les travaux réalisés ont permis de mettre progressivement en place les différentes briques constituant le système, depuis l'acquisition des données jusqu'à leur visualisation. Une première étape a consisté à développer et à valider le fonctionnement du nœud capteur autour du microcontrôleur STM32L476RG. Les différents capteurs utilisés permettent notamment de mesurer la température, l'humidité, le poids de la ruche ainsi que la détection de mouvement.

La communication sans fil a ensuite été mise en œuvre à l'aide du module Wio-E5-HF. Après une première étude de la communication LoRa, le système a été configuré afin d'établir une communication LoRaWAN avec The Things Network (TTN). Deux ruches ont ainsi pu être connectées à la même infrastructure réseau tout en permettant de différencier leurs données.

La chaîne de récupération et de visualisation des données a également été mise en place. Les données reçues par TTN sont récupérées par MQTT dans Node-RED, puis transmises vers InfluxDB pour leur stockage. Grafana est ensuite utilisé pour représenter graphiquement l'évolution des différentes grandeurs mesurées.

Enfin, une première étude énergétique du système a été réalisée afin d'estimer la consommation électrique d'une ruche et de déterminer les caractéristiques nécessaires d'une solution d'alimentation autonome basée sur une batterie et un panneau photovoltaïque.

Le tableau ?? présente les principaux travaux réalisés au cours du stage.

8.2 Difficultés rencontrées

La réalisation du projet a nécessité la résolution de plusieurs difficultés, notamment lors de la mise en place de la communication sans fil et de l'intégration des différents composants matériels et logiciels.

Une première difficulté concernait la configuration de l'environnement LoRaWAN. La mise en place de la gateway SenseCAP et son association à The Things Network ont nécessité plusieurs étapes de configuration. La gestion des identifiants des équipements, notamment les identifiants EUI, a également nécessité une attention particulière afin de permettre la connexion correcte des équipements au réseau.

Une autre difficulté concernait l'intégration des deux ruches dans l'infrastructure LoRaWAN. Une seconde application et un second équipement ont été configurés sur TTN afin

de pouvoir identifier séparément les données provenant de chaque ruche.

La mise en place de la chaîne de traitement des données a également demandé plusieurs étapes de configuration. Node-RED devait récupérer correctement les messages MQTT provenant de TTN, extraire les différentes valeurs du `decoded_payload`, puis transmettre ces informations à InfluxDB sous un format adapté. Grafana a ensuite été configuré afin d’interroger la base de données et de représenter les mesures sous forme de séries temporelles.

Enfin, l’étude énergétique a nécessité de mesurer la consommation réelle du système afin d’éviter de dimensionner la solution d’alimentation uniquement à partir des caractéristiques théoriques des composants.

8.3 Bilan énergétique

Les mesures réalisées à l’aide du moniteur de consommation USB ont montré que la ruche consomme environ 0,51 W en régime permanent, pour une tension d’environ 5,093 V et un courant de 98,3 mA.

La consommation quotidienne peut ainsi être estimée à :

$$E_{\text{jour}} = P \times 24 \quad (8.1)$$

soit :

$$E_{\text{jour}} = 0,51 \times 24 = 12,24 \text{ Wh/jour.} \quad (8.2)$$

Sur une période de 30 jours, la consommation correspondante est d’environ :

$$E_{\text{mois}} = 12,24 \times 30 = 367,2 \text{ Wh/mois.} \quad (8.3)$$

Pour assurer une autonomie de trois jours sans alimentation extérieure, la capacité énergétique minimale nécessaire est estimée à :

$$E_{3j} = 12,24 \times 3 = 36,72 \text{ Wh.} \quad (8.4)$$

En prenant en compte une limitation de la profondeur de décharge de la batterie, une capacité supérieure à cette valeur est nécessaire. Dans l’hypothèse d’une batterie lithium-ion dont 80 % de la capacité peut être utilisée, la capacité minimale estimée est d’environ 45,9 Wh.

Cette étude a conduit à envisager une batterie de 12 V associée à un convertisseur permettant d’alimenter le système en 5 V. Pour une autonomie de plusieurs jours, une batterie de l’ordre de 12 V et 5 à 6 Ah a été envisagée.

Concernant la production photovoltaïque, une première estimation a conduit à retenir un panneau solaire d’au moins 20 W afin de disposer d’une marge suffisante pendant les périodes hivernales à Toulouse. Un panneau de 30 W pourrait également être envisagé afin d’augmenter la marge énergétique du système.

Ces valeurs constituent toutefois un premier dimensionnement. Une validation expérimentale sur plusieurs jours et dans différentes conditions météorologiques serait nécessaire pour confirmer le dimensionnement final.

8.4 Perspectives

Plusieurs améliorations peuvent être envisagées afin de poursuivre le développement du système.

La première perspective consiste à poursuivre l'optimisation de la consommation énergétique du nœud capteur. La consommation mesurée en régime permanent reste relativement importante pour une application fonctionnant sur batterie et panneau solaire. La mise en place de modes basse consommation du microcontrôleur et la réduction de la fréquence des transmissions LoRaWAN pourraient permettre d'améliorer significativement l'autonomie.

Une deuxième perspective concerne l'amélioration de la solution d'alimentation. Le dimensionnement actuel du panneau photovoltaïque et de la batterie devra être validé expérimentalement afin de vérifier que le système reste autonome pendant plusieurs jours, notamment durant les périodes de faible ensoleillement.

La chaîne de supervision pourrait également être enrichie. Des mécanismes d'alerte pourraient par exemple être ajoutés à Node-RED afin de prévenir l'utilisateur lorsqu'une grandeur dépasse un seuil défini. Cela pourrait concerner notamment le poids de la ruche, la température, l'humidité ou encore la détection d'un mouvement inhabituel.

Une autre perspective importante concerne le décodage et l'analyse des trames LoRa. L'étude pourrait être approfondie afin de mieux comprendre la structure des transmissions et d'analyser les différents paramètres du signal radio.

Enfin, le système pourrait être amélioré sur le plan matériel par l'intégration des différents éléments sur une carte électronique dédiée. Cette évolution permettrait de réduire l'encombrement du système, d'améliorer sa robustesse et de faciliter son intégration dans une ruche réelle.

8.5 Bilan général

Ce stage a permis de mettre en œuvre une chaîne complète de communication IoT reposant sur des systèmes embarqués et la technologie LoRaWAN. Le projet a couvert plusieurs domaines complémentaires : acquisition de données par capteurs, programmation d'un microcontrôleur STM32, communication sans fil, configuration d'une infrastructure LoRaWAN, traitement des données et visualisation.

Les travaux réalisés constituent ainsi une base pour poursuivre l'optimisation du système, notamment en matière de consommation énergétique, d'autonomie, de robustesse et d'analyse des communications LoRaWAN.

Bibliographie

- [1] STS SN-EC, *Caractérisation de l'interface radio LoRa d'un réseau de communication LoRaWAN*, Lycée Dorian, conférence du 2 mai 2018.
- [2] *The Hidden Side of LoRa*, Disk91, 19 août 2024. Disponible sur : <https://www.disk91.com/2024/technology/lora/the-hidden-side-of-lora/>.
- [3] MONTAGNY, Sylvain, *Tout ce que vous devez savoir sur le LoRaWAN en 40 mins*, Université Savoie Mont Blanc, YouTube, 3 juin 2022. Disponible sur : <https://www.youtube.com/watch?v=j00NEdk0m28>. Consulté le 24 août 2026.

A. Annexe — Fonctions de décodage Node-RED et TTN

A.1 Décodage des données reçues sur TTN

Listing A.1 – Fonction TTN - Décodeur TTN

```
1 function decodeUplink(input) {
2   // Convert bytes to ASCII string
3   var message = "";
4   for (var i = 0; i < input.bytes.length; i++) {
5     message += String.fromCharCode(input.bytes[i]);
6   }
7
8
9   // Expected format: "P:0.000 Temp_int:23.4 Humid_int:40.0 1"
10  var poids = null;
11  var temp_int = null;
12  var humid_int = null;
13  var temp_ext = null;
14  var humid_ext = null;
15  var motion = null;
16  var tension = null;
17  // Extract Poids
18  var poidsMatch = message.match(/P:([\d.]+)/);
19  if (poidsMatch) poids = parseFloat(poidsMatch[1]);
20
21  // Extract Temp_int
22  var tempMatch = message.match(/Tin:([\d.]+)/);
23  if (tempMatch) temp_int = parseFloat(tempMatch[1]);
24
25  // Extract Humid_int
26  var humidMatch = message.match(/Hin:([\d.]+)/);
27  if (humidMatch) humid_int = parseFloat(humidMatch[1]);
28
29  // Extract Temp_ext
30  var tempMatch1 = message.match(/Tout:([\d.]+)/);
31  if (tempMatch1) temp_ext = parseFloat(tempMatch1[1]);
32
33  // Extract Humid_ext
34  var humidMatch1 = message.match(/Hout:([\d.]+)/);
35  if (humidMatch1) humid_ext = parseFloat(humidMatch1[1]);
36
37  // Extract Tension
38  var tensionMatch = message.match(/Vin:([\d.]+)/);
39  if (tensionMatch) tension = parseFloat(tensionMatch[1]);
```

```
40
41 // Extract motion (last number 0 or 1)
42 var motionMatch = message.match(/s([01])$/);
43 if (motionMatch) motion = parseInt(motionMatch[1]);
44
45 return {
46   data: {
47     tension_V: tension,
48     poids_kg: poids,
49     temperature_interieure: temp_int,
50     humidite_interieure: humid_int,
51     temperature_exterieur: temp_ext,
52     humidite_exterieur: humid_ext,
53     mouvement: motion
54   },
55   warnings: [],
56   errors: []
57 };
58 }
```

A.2 Extraction de la tension batterie

La fonction suivante, insérée dans un nœud **Function** entre le nœud **MQTT In** et le nœud **InfluxDB Out** correspondant, extrait la tension batterie du message décodé par TTN :

Listing A.2 – Fonction Node-RED - extraction de la tension batterie

```
1 const val = msg.payload['uplink_message']['decoded_payload']['tension_V'];
2 if (val === null || val === undefined) return null;
3 msg.payload = val;
4 return msg;
```

Le champ `tension_V` est lu dans la structure `decoded_payload` du message MQTT reçu de TTN. Le test de nullité (`null/undefined`) évite l'écriture d'un point de donnée vide dans InfluxDB lorsque le champ est absent d'une trame donnée retourner `null` à ce stade stoppe le flux Node-RED pour ce message, sans déclencher l'écriture en base.

A.3 Extraction de la valeur de détection de mouvement

Listing A.3 – Fonction Node-RED - extraction de la valeur de détection de mouvement

```
1 const val = msg.payload['uplink_message']['decoded_payload']['mouvement'];
2 if (val === null || val === undefined) return null;
3 msg.payload = val;
4 return msg;
```

A.4 Extraction de la valeur de détection de la masse

Listing A.4 – Fonction Node-RED - extraction de la masse

```
1 const val = msg.payload['uplink_message']['decoded_payload']['  
  poids_kg'];  
2 if (val === null || val === undefined) return null;  
3 msg.payload = val;  
4 return msg;
```

A.5 Extraction de la valeur de détection de l'humidité

Listing A.5 – Fonction Node-RED - extraction de l'humidité

```
1 const val = msg.payload['uplink_message']['decoded_payload']['  
  humidite_exterieur'];  
2 if (val === null || val === undefined) return null;  
3 msg.payload = val;  
4 return msg;
```